

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\agent-a4fc4c6339e15ad84.jsonl`  
- SHA-256: `cebc24d882e2e869f47fd44c2f375c082621ef8f32c1e218d942ebd6fe78a012`  
- Source modified: `2026-06-14T08:27:00+00:00`  
- Imported at: `2026-07-05T16:48:25+00:00`  
- project: `subagents`  
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`
```

Transcript

1. user (2026-06-14T08:23:08.499Z)

Implement a focused, mergeable engineering PR in the badminton-highlight-indexer repo (cwd C:/Users/avidu/Projects/badminton-highlight-indexer): wire the run-telemetry black box (backend/utils/run_telemetry.py, merged in #134) into the fusion_golden extractor so long GPU runs auto-keep a forensic ~10KB runlog. NO other changes (no audio work).

BRANCH SAFETY (critical ? concurrent sessions share this checkout): work in an ISOLATED git worktree.

```
git -C C:/Users/avidu/Projects/badminton-highlight-indexer fetch origin  
git -C C:/Users/avidu/Projects/badminton-highlight-indexer worktree add  
C:/Users/avidu/Projects/bhi-telwire -b feat/telemetry-wire-fusion-golden origin/master  
Do ALL edits in that worktree dir. Before committing: `git log --oneline origin/master..HEAD`  
must list ONLY your commit; stage explicit paths (never `git add -A`).
```

WIRE-IN (backend/eval/fusion_golden.py) ? additive, default-ON but fully graceful:

```
- In run(): build `telem = RunTelemetry(os.path.join(out_dir, "run_telemetry.jsonl"),  
label="fusion_golden")` (import from backend.utils.run_telemetry). Emit  
`telem.event("video_start", spec["name"])` before each extract_video; on success  
`telem.event("video_done", spec["name"], windows=len(data["candidates"]), positives=n_pos)`;  
wrap the per-video work in try/except that does `telem.event("error", spec["name"], err=str(e))`  
then re-raises (so a death records GPU/CPU state). Pass telem into extract_video.  
- In extract_video(spec, ..., telemetry=None): inside the existing `_progress(done, total)`  
callback, add `if telemetry is not None: telemetry.snapshot(done, total, phase=spec["name"])`.  
- A telemetry hiccup must NEVER break extraction (run_telemetry is already internally safe; just  
ensure the wiring itself can't raise ? e.g. guard the RunTelemetry construction).  
- Keep existing tests green: test_fusion_golden_run_is_resumable monkeypatches extract_video, so  
run() still must work (the extra run_telemetry.jsonl write into out_dir is harmless).  
- Also update scratch/extract_until_done.ps1 if trivial (it's gitignored ? do NOT commit it;  
optional).
```

TEST (add tests/test_fusion_golden_telemetry.py): with a stubbed extract_video, run() writes output/run_telemetry.jsonl containing a meta line + "video_start"/"video_done" event lines; and a telemetry-collector failure (inject a raising gpu_fn via monkeypatching, or rely on graceful degradation) does NOT break run(). Use RunTelemetry's injectable gpu_fn/sys_fn/clock for determinism (no real GPU).

VERIFY in the worktree: `python -m ruff check backend/eval/fusion_golden.py
tests/test_fusion_golden_telemetry.py`; `python -m mypy backend/eval/fusion_golden.py`; `python
-m pytest tests/test_fusion_compare.py tests/test_fusion_golden_telemetry.py -q`. All must pass.

PR: commit (explicit paths; end the message with `Co-Authored-By: Claude Opus 4.8 (1M context)
<noreply@anthropic.com>`, `git push -u origin feat/telemetry-wire-fusion-golden`, `gh pr create
--base master`. DO NOT merge ? leave it for review. Leave the worktree in place and report its
path.

Return: the PR number+URL, files changed, the exact ruff/mypy/pytest results, and the worktree

path.

2. assistant (2026-06-14T08:23:10.509Z)

I'll implement this PR. Let me start by setting up the isolated worktree and understanding the codebase.

3. user (2026-06-14T08:23:13.456Z)

(Bash completed with no output)

4. user (2026-06-14T08:23:13.464Z)

```
1      """Structured run telemetry ? a compact, rotating JSONL "black box" for long/flaky runs.
2
3      A long GPU/CV run (the fusion-golden extraction, a WSL/WASB pass, an ffmpeg stitch) can
die with
4      **no Python traceback** ? the OS kills it (a console-control event, an OOM, a thermal
event). When
5      that happens the only forensic evidence is whatever was last flushed to disk. This
module keeps a
6      tiny (~10 KB) rotating runlog of *execution velocity + machine health* so a post-mortem
is a
7      bullseye instead of a guess: "it was at 26 fps, GPU 85 °C throttled to 1125/2100 MHz,
4.8 GB VRAM
8      free, CPU 60 %, RAM 21/32 GB ? then vanished."
9
10     It is the **observability** companion to the `ExecutionPolicy` *control* layer
11     (`docs/EXECUTION_POLICY.md`): the policy decides wait / abort / resume; this records the
signals
12     that explain *why* a run slowed or died, and lets us compare execution velocity across
quality
13     iterations.
14
15     Design:
16     - **Caller-driven snapshots** (no background thread): call ``snapshot(done, total)``
from an
17     existing progress callback; fps is derived from the delta since the previous snapshot.
18     ``event(kind, msg)`` records discrete moments (start, video-done, error) WITH a health
sample, so
19     the last line before a kill captures the machine state at death.
20     - **Capped + rotating** : a fixed ``meta`` header line (machine / GPU / start) is always
kept; recent
21     snapshot/event lines form an on-disk ring trimmed to ``max_bytes`` (~10 KB). The file
is rewritten
22     atomically (tmp + ``os.replace``) on each write, so a hard kill never leaves it half-
written.
23     - **Graceful degradation** : GPU stats come from ``nvidia-smi`` (absent -> ``None``);
system stats
24     from ``psutil`` (not installed -> ``None``). Collection never raises into the caller ?
a telemetry
25     hiccup must not break the run it observes.
26     - **Deterministic / testable** : the clock and the two collectors are injectable, so fps,
rotation,
27     and parsing are unit-tested with no real GPU, no psutil, and no wall-clock.
28
29     All stdlib + optional ``psutil`` (BSD-3, commercial-clean). No network, no training-data
path.
30     """
31     from __future__ import annotations
32
33     import json
34     import os
35     import shutil
```

```

36     import subprocess
37     import time
38     from typing import Any, Callable, Dict, List, Optional
39
40     GpuFn = Callable[[], Optional[Dict[str, Any]]]
41     SysFn = Callable[[], Optional[Dict[str, Any]]]
42
43     # nvidia-smi fields pulled in one query, in this order (see gpu_stats parsing).
44     _GPU_QUERY =
"temperature.gpu,utilization.gpu,power.draw,clocks.sm,clocks.max.sm,memory.used,memory.free"
45     _THROTTLE_CLOCK_FRAC = 0.8 # SM clock below this fraction of max => flagged throttled
46     _THROTTLE_TEMP_C = 84.0 # ... or temperature at/above this (laptop Max-Q throttle
territory)
47
48
49     def _num(s: str) -> Optional[float]:
50         try:
51             return float(s)
52         except (TypeError, ValueError):
53             return None
54
55
56     def gpu_stats(timeout: float = 4.0) -> Optional[Dict[str, Any]]:
57         """One ``nvidia-smi`` sample as a dict, or ``None`` if nvidia-smi is unavailable /
errors.
58
59         Returns util %, temperature, power (W), current + max SM clock (MHz), VRAM used /
free (MB), and a
60         derived ``throttled`` flag (SM clock well below max, or hot). Pure read; never
raises.
61
62         """
63         exe = shutil.which("nvidia-smi")
64         if not exe:
65             return None
66         try:
67             out = subprocess.run(
68                 [exe, f"--query-gpu={_GPU_QUERY}", "--format=csv,noheader,nounits"],
69                 capture_output=True, text=True, timeout=timeout, check=True,
70             ).stdout.strip()
71         except (subprocess.SubprocessError, OSError):
72             return None
73         row = out.splitlines()[0] if out else ""
74         parts = [p.strip() for p in row.split(",")]
75         if len(parts) < 7:
76             return None
77         vals = [_num(p) for p in parts[:7]]
78         temp, util, power, sm, sm_max, used, free = vals
79         throttled = bool(temp is not None and temp >= _THROTTLE_TEMP_C)
80         if sm is not None and sm_max:
81             throttled = throttled or sm < _THROTTLE_CLOCK_FRAC * sm_max
82         return {
83             "util_pct": util, "temp_c": temp, "power_w": power,
84             "sm_mhz": sm, "sm_max_mhz": sm_max,
85             "vram_used_mb": used, "vram_free_mb": free,
86             "throttled": throttled,
87         }
88
89     def system_stats() -> Optional[Dict[str, Any]]:
90         """CPU % + RAM via ``psutil``, or ``None`` if psutil isn't installed. Never
raises."""
91         try:
92             import psutil # optional dep (BSD-3); only used when present
93         except ImportError:
94             return None

```

```

95         try:
96             vm = psutil.virtual_memory()
97             return {
98                 "cpu_pct": psutil.cpu_percent(interval=None),
99                 "ram_used_gb": round(vm.used / 1e9, 2),
100                 "ram_total_gb": round(vm.total / 1e9, 2),
101                 "ram_pct": vm.percent,
102             }
103         except Exception: # noqa: BLE001 - psutil edge cases must not break the run
104             return None
105
106
107     def gpu_name() -> Optional[str]:
108         """The GPU model string (for the runlog header), or ``None`` without nvidia-smi."""
109         exe = shutil.which("nvidia-smi")
110         if not exe:
111             return None
112         try:
113             out = subprocess.run([exe, "--query-gpu=name", "--format=csv,noheader"],
114                                 capture_output=True, text=True, timeout=4.0,
115                                 check=True).stdout
116         except (subprocess.SubprocessError, OSError):
117             return None
118         return out.splitlines()[0].strip() if out else None
119
120     class RunTelemetry:
121         """A small rotating JSONL runlog of execution velocity + machine health.
122
123         Call ``snapshot(done, total, phase=...)`` from a progress callback and ``event(kind,
124         msg)`` at
125         milestones. The file holds a ``meta`` header line plus the most recent snapshot /
126         event lines
127         that fit within ``max_bytes`` (~10 KB), rewritten atomically on every write.
128         """
129
130         def __init__(self, path: str, label: str = "run", max_bytes: int = 10_000,
131                     gpu_fn: GpuFn = gpu_stats, sys_fn: SysFn = system_stats,
132                     clock: Callable[[], float] = time.time) -> None:
133             self.path = path
134             self.label = label
135             self.max_bytes = max_bytes
136             self._gpu_fn = gpu_fn
137             self._sys_fn = sys_fn
138             self._clock = clock
139             self._t0 = clock()
140             self._last_done: Optional[int] = None
141             self._last_t: Optional[float] = None
142             self._lines: List[str] = [] # recent body lines (on-disk ring, trimmed to
143             max_bytes)
144
145             self._meta = json.dumps({"meta": {
146                 "label": label, "started": self._iso(self._t0), "pid": os.getpid(),
147                 "gpu_name": gpu_name(), "cpu_count": os.cpu_count(),
148             }}, separators=(",", ":"))
149             self._write()
150
151         # -- public API -----
152         def snapshot(self, done: Optional[int] = None, total: Optional[int] = None,
153                     phase: Optional[str] = None, **extra: Any) -> Dict[str, Any]:
154             """Record a velocity + health sample; ``fps`` is derived from the previous
155             snapshot."""
156             now = self._clock()
157             rec: Dict[str, Any] = {"t": self._iso(now), "elapsed_s": round(now - self._t0,
158                                     1)}
159             if phase is not None:

```

```

154         rec["phase"] = phase
155     if done is not None:
156         rec["done"] = done
157         if total:
158             rec["total"] = total
159             rec["pct"] = round(100.0 * done / total, 1)
160             if self._last_done is not None and self._last_t is not None and now >
self._last_t:
161                 rec["fps"] = round((done - self._last_done) / (now - self._last_t), 1)
162                 self._last_done, self._last_t = done, now
163         rec.update(extra)
164         self._attach_health(rec)
165         self._append(rec)
166         return rec
167
168     def event(self, kind: str, msg: str = "", **extra: Any) -> Dict[str, Any]:
169         """Record a milestone (start, video_done, error, restart) WITH a health
sample."""
170         now = self._clock()
171         rec: Dict[str, Any] = {"t": self._iso(now), "elapsed_s": round(now - self._t0,
1),
172                                "event": kind}
173         if msg:
174             rec["msg"] = msg
175             rec.update(extra)
176             self._attach_health(rec)
177             self._append(rec)
178             return rec
179
180     # -- internals -----
181     def _attach_health(self, rec: Dict[str, Any]) -> None:
182         gpu = self._safe(self._gpu_fn)
183         sys_ = self._safe(self._sys_fn)
184         if gpu is not None:
185             rec["gpu"] = gpu
186         if sys_ is not None:
187             rec["sys"] = sys_
188
189     def _append(self, rec: Dict[str, Any]) -> None:
190         self._lines.append(json.dumps(rec, separators=(",", ":")))
191         budget = self.max_bytes - len(self._meta) - 1
192         while len(self._lines) > 1 and sum(len(s) + 1 for s in self._lines) > budget:
193             self._lines.pop(0) # drop oldest body line; the meta header is always kept
194         self._write()
195
196     def _write(self) -> None:
197         try:
198             tmp = self.path + ".tmp"
199             with open(tmp, "w", encoding="utf-8") as f:
200                 f.write(self._meta + "\n")
201                 for s in self._lines:
202                     f.write(s + "\n")
203             os.replace(tmp, self.path)
204         except OSError:
205             pass # telemetry must never break the run it observes
206
207     @staticmethod
208     def _safe(fn: GpuFn) -> Optional[Dict[str, Any]]:
209         try:
210             return fn()
211         except Exception: # noqa: BLE001 - a collector hiccup must not break the run
212             return None
213
214     @staticmethod
215     def _iso(t: float) -> str:

```

```
216         return time.strftime("%Y-%m-%dT%H:%M:%S", time.localtime(t))
217
```

5. assistant (2026-06-14T08:23:15.758Z)

Now let me set up the worktree and read the target file.

6. user (2026-06-14T08:23:17.665Z)

Preparing worktree (new branch 'feat/telemetry-wire-fusion-golden')
branch 'feat/telemetry-wire-fusion-golden' set up to track 'origin/master'.
HEAD is now at ec5bbe0 chore(eval): track experiment_leaderboard.json (fusion P3/P4 honest LOVO records) (#139)

7. user (2026-06-14T08:23:19.889Z)

```
1      """Offline golden-set person-feature extractor ? the fusion-P3 measurement's GPU step.
2
3      Mirrors `distill_local`'s CSV-driven flow but ADDS the ~5 person-cue features per
shuttle
4      candidate window, by running `PersonDetector` over the ORIGINAL video. It reuses the
5      EXISTING golden trajectory CSVs (so **no WASB re-run**), needs **no Gemini and no DB**,
and
6      writes one small JSON per video. The LOVO comparison (`fusion_compare.py`) then runs
100% on
7      CPU offline. So the GPU touches exactly one thing ? person detection ? and the lift is
8      measured by the shipped experiment harness; person features enter only as feature
columns the
9      learned scorer weights (no special-casing, default-OFF until the golden ?F1 justifies
them).
10
11      Run (on the GPU box):
12      python -m backend.eval.fusion_golden --manifest output/human_lovo_manifest.json
--out-dir output
13
14      Each output `output/<name>_golden_features.json` is the replayable feature substrate ?
re-run
15      `fusion_compare` with different variants/thresholds forever without re-detecting on the
GPU.
16      """
17      from __future__ import annotations
18
19      import argparse
20      import json
21      import os
22      import time
23      from typing import Any, Dict, List, Optional
24
25      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
26      from backend.pipeline.detectors import rally_gate
27      from backend.pipeline.detectors import fusion_features as ff
28      from backend.eval import distill_local
29      from backend.eval.calibrate_wasb import load_golden_gts
30      from backend.eval.training import label_candidates
31
32      # The 5 fps/resolution-invariant shuttle features distill_local already computes.
33      SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
34      # The 5 person-cue features (fusion_features.PERSON_FEATURE_NAMES).
35      PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)
36
37
38      def _derive_video_path(spec: Dict[str, Any]) -> str:
39          """Original MP4 for a manifest entry ? explicit `video_path`, else derived from the
golden labels path (`X.rallies.csv` -> `X.MP4`, the golden-set naming
```

```

convention)."""
41     if spec.get("video_path"):
42         return str(spec["video_path"])
43     labels = str(spec["labels"])
44     if labels.endswith(".rallies.csv"):
45         return labels[: -len(".rallies.csv")] + ".MP4"
46     raise ValueError(f"{spec.get('name')}: no video_path and labels {labels!r} isn't
*.rallies.csv")
47
48
49     def extract_video(spec: Dict[str, Any], detector: Optional[Any] = None,
50                     frame_skip: int = 3, iou: float = 0.5,
51                     log_every_sec: float = 15.0) -> Dict[str, Any]:
52         """One golden video -> a feature record (shuttle + person features per candidate
window).
53
54         Returns ``{name, fps, frame_width,
candidates: [{start, end, shuttle {}, person {}, label}], gts}``.
55         `detector` (anything with `detections_over_windows`) is injectable for GPU-free
tests;
56         when None a real `PersonDetector` is built (loads torchvision on first use).
57         """
58         fps, fw = float(spec["fps"]), float(spec["frame_width"])
59         traj = str(spec["trajectory"])
60         points = TrackNetRunner.parse_trajectory_csv(traj)
61         intervals, x_shuttle = distill_local.build_candidates(traj, fps, fw)
62         gts = load_golden_gts(str(spec["labels"]))
63         labels = label_candidates(intervals, gts, iou)
64         # Net axis for the person two-sided split ? reuse rally_gate's camera-agnostic
estimate
65         # (parity with the shuttle net-crossing gate) rather than hard-coding an
orientation.
66         axis, net_px, _span = rally_gate.estimate_play_axis(points)
67         video_path = _derive_video_path(spec)
68
69         if detector is None:
70             from backend.pipeline.detectors.person_detector import PersonDetector
71             detector = PersonDetector({"indexing": {"use_gpu": True}})
72
73         # Single-pass person detection over ALL candidate windows (open the video ONCE, read
74         # forward ? no per-window seek). Then compute features per window (pure, no GPU).
75         n = len(intervals)
76         win_frames = [(round(s * fps), round(e * fps)) for (s, e) in intervals]
77         print(f"[fusion_golden] {spec['name']}: {n} windows ? person-detecting in one
pass...", flush=True)
78         t0 = time.monotonic()
79         state = {"last": t0}
80
81         def _progress(done: int, total: int) -> None:
82             now = time.monotonic()
83             if now - state["last"] >= log_every_sec or done >= total:
84                 el = now - t0
85                 rate = done / el if el > 0 else 0.0
86                 eta = (total - done) / rate if rate > 0 else 0.0
87                 pct = 100 * done // total if total else 0
88                 print(f"[fusion_golden] {spec['name']}: detect frame {done}/{total}
({pct}%) "
89                       f"| {el:.0f}s elapsed | ETA {eta:.0f}s", flush=True)
90                 state["last"] = now
91
92         per_window = detector.detections_over_windows(video_path, win_frames, frame_skip,
93                                                       progress_cb=_progress)
94
95         candidates: List[Dict[str, Any]] = []
96         for i, ((s, e), xs) in enumerate(zip(intervals, x_shuttle)):

```

```

97         det = per_window[i]
98         det_fw = det.get("frame_width") or fw
99         det_fh = det.get("frame_height") or 0.0
100         # Normalise the (pixel) net coord to 0-1 on the play axis, matching the foot-
point
101         # normalisation fusion_features uses (x/width, y/height).
102         if axis == "x":
103             net_norm = net_px / det_fw if det_fw > 0 else 0.5
104         else:
105             net_norm = net_px / det_fh if det_fh > 0 else 0.5
106         sf, ef = win_frames[i]
107         shuttle_in = [p for p in points if sf <= p.frame <= ef]
108         person = ff.compute_person_features(det["frames"], shuttle_in, det_fw, det_fh,
109                                           court_polygon=None, net=(axis, net_norm))
110         candidates.append({
111             "start": float(s), "end": float(e),
112             "shuttle": {k: float(v) for k, v in zip(SHUTTLE_FEATURE_NAMES, xs)},
113             "person": {k: float(v) for k, v in person.items()},
114             "label": int(labels[i]),
115         })
116     return {
117         "name": spec["name"], "fps": fps, "frame_width": fw,
118         "candidates": candidates,
119         "gts": [[float(a), float(b)] for a, b in gts],
120     }
121
122
123 def run(manifest_path: str, out_dir: str, frame_skip: int = 3, iou: float = 0.5,
124        fresh: bool = False, smallest_first: bool = False) -> List[str]:
125     with open(manifest_path, encoding="utf-8") as f:
126         specs = json.load(f)
127     if smallest_first:
128         # Process small videos first for quick feedback (the slow full match lands
last);
129         # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
row/frame).
130         specs = sorted(specs, key=lambda s: (os.path.getsize(s["trajectory"])
131                                             if os.path.isfile(s.get("trajectory", ""))
else 0))
132     os.makedirs(out_dir, exist_ok=True)
133     written: List[str] = []
134     for vi, spec in enumerate(specs, 1):
135         out = os.path.join(out_dir, f"{spec['name']}_golden_features.json")
136         # Resumable: skip a video whose JSON already exists (a re-run picks up where a
prior
137         # run left off instead of re-detecting). --fresh forces a redo.
138         if not fresh and os.path.isfile(out):
139             print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: already
extracted "
140                  f"-> {out} (use --fresh to redo)", flush=True)
141             written.append(out)
142             continue
143         print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: extracting...",
flush=True)
144         t0 = time.monotonic()
145         data = extract_video(spec, frame_skip=frame_skip, iou=iou)
146         n_pos = sum(c["label"] for c in data["candidates"])
147         with open(out, "w", encoding="utf-8") as f:
148             json.dump(data, f, indent=2)
149         print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: DONE in
{time.monotonic() - t0:.0f}s "
150              f"| {len(data['candidates'])} windows ({n_pos} positive) |
{len(data['gts'])} golden rallies "
151              f"-> {out}", flush=True)
152         written.append(out)

```



```

153         return written
154
155
156     def main() -> None:
157         ap = argparse.ArgumentParser(description="Extract golden-set shuttle+person features
(fusion P3 GPU step).")
158         ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
159         ap.add_argument("--out-dir", default="output")
160         ap.add_argument("--frame-skip", type=int, default=3)
161         ap.add_argument("--iou", type=float, default=0.5)
162         ap.add_argument("--fresh", action="store_true", help="re-extract even if a JSON
already exists")
163         ap.add_argument("--smallest-first", action="store_true",
164                         help="process smaller videos first for quick feedback (LOVO is
order-independent)")
165         args = ap.parse_args()
166         run(args.manifest, args.out_dir, args.frame_skip, args.iou, fresh=args.fresh,
167             smallest_first=args.smallest_first)
168
169
170     if __name__ == "__main__":
171         main()
172

```

8. assistant (2026-06-14T08:23:22.495Z)

Now let me look at the existing test for resumability and existing telemetry tests to understand patterns.

9. user (2026-06-14T08:23:23.734Z)

```

Exit code 127
/usr/bin/bash: line 1: Select-String: command not found

```

10. user (2026-06-14T08:23:26.345Z)

No files found

11. user (2026-06-14T08:23:29.523Z)

```

test_fusion_compare.py
test_fusion_features.py
test_fusion_hybrid.py
test_run_telemetry.py

```

12. assistant (2026-06-14T08:23:31.975Z)

The existing `test\_fusion\_golden\_run\_is\_resumable` test was mentioned ? let me find where it lives.

13. user (2026-06-14T08:23:33.215Z)

```

1     """GPU-free, psutil-free tests for the run-telemetry black box.
2
3     The nvidia-smi / psutil collectors and the clock are all injectable, so fps, rotation,
atomic
4     JSONL writing, and graceful degradation are verified with no real GPU and no wall-clock.
5     """
6     import json
7     from types import SimpleNamespace
8
9     from backend.utils import run_telemetry as rt
10
11

```

```

12 # ----- gpu_stats parsing
13 def test_gpu_stats_parses_and_flags_throttle(monkeypatch):
14     monkeypatch.setattr(rt.shutil, "which", lambda _: "nvidia-smi")
15     monkeypatch.setattr(rt.subprocess, "run",
16                         lambda *a, **k: SimpleNamespace(stdout="85, 81, 79.18, 1125,
2100, 3000, 4900\n"))
17     g = rt.gpu_stats()
18     assert g is not None
19     assert g["temp_c"] == 85.0 and g["util_pct"] == 81.0 and g["power_w"] == 79.18
20     assert g["sm_mhz"] == 1125.0 and g["sm_max_mhz"] == 2100.0
21     assert g["vram_used_mb"] == 3000.0 and g["vram_free_mb"] == 4900.0
22     assert g["throttled"] is True # 1125 < 0.8*2100 AND 85 >= 84
23
24
25 def test_gpu_stats_not_throttled(monkeypatch):
26     monkeypatch.setattr(rt.shutil, "which", lambda _: "nvidia-smi")
27     monkeypatch.setattr(rt.subprocess, "run",
28                         lambda *a, **k: SimpleNamespace(stdout="60, 50, 90.0, 1950,
2100, 2000, 6000"))
29     assert rt.gpu_stats()["throttled"] is False # 1950 >= 1680 and 60 < 84
30
31
32 def test_gpu_stats_none_without_nvidia_smi(monkeypatch):
33     monkeypatch.setattr(rt.shutil, "which", lambda _: None)
34     assert rt.gpu_stats() is None
35
36
37 def test_gpu_stats_none_on_error(monkeypatch):
38     monkeypatch.setattr(rt.shutil, "which", lambda _: "nvidia-smi")
39
40     def _boom(*a, **k):
41         raise rt.subprocess.TimeoutExpired(cmd="nvidia-smi", timeout=4)
42
43     monkeypatch.setattr(rt.subprocess, "run", _boom)
44     assert rt.gpu_stats() is None
45
46
47 def test_system_stats_shape():
48     r = rt.system_stats() # real psutil if present, else None ? both are valid
49     assert r is None or {"cpu_pct", "ram_used_gb", "ram_total_gb", "ram_pct"} <= set(r)
50
51
52 # ----- RunTelemetry
53 class _Clock:
54     def __init__(self, t=1000.0):
55         self.t = t
56
57     def __call__(self):
58         return self.t
59
60     def tick(self, dt):
61         self.t += dt
62
63
64 def _fake_gpu():
65     return {"temp_c": 85.0, "util_pct": 80.0, "throttled": True}
66
67
68 def _fake_sys():
69     return {"cpu_pct": 50.0, "ram_used_gb": 21.0}
70
71
72 def _no_gpu_name(monkeypatch):
73     # keep RunTelemetry construction hermetic (no real nvidia-smi for the header)
74     monkeypatch.setattr(rt, "gpu_name", lambda: "FakeGPU")

```

```

75
76
77 def test_snapshot_fps_pct_and_jsonl(tmp_path, monkeypatch):
78     _no_gpu_name(monkeypatch)
79     clk = _Clock()
80     path = str(tmp_path / "rt.jsonl")
81     t = rt.RunTelemetry(path, label="x", gpu_fn=_fake_gpu, sys_fn=_fake_sys, clock=clk)
82     clk.tick(10)
83     t.snapshot(done=100, total=1000, phase="vidA")
84     clk.tick(10)
85     rec = t.snapshot(done=200, total=1000, phase="vidA")
86     assert rec["fps"] == 10.0 and rec["pct"] == 20.0
87     assert rec["gpu"]["temp_c"] == 85.0 and rec["sys"]["cpu_pct"] == 50.0
88     lines = [ln for ln in open(path, encoding="utf-8").read().splitlines() if ln]
89     assert json.loads(lines[0])["meta"]["label"] == "x" # header first
90     for ln in lines[1:]:
91         json.loads(ln) # every body line is valid
92
93 JSON
94
95 def test_event_captures_health_at_death(tmp_path, monkeypatch):
96     _no_gpu_name(monkeypatch)
97     path = str(tmp_path / "rt.jsonl")
98     t = rt.RunTelemetry(path, gpu_fn=_fake_gpu, sys_fn=_fake_sys, clock=_Clock())
99     t.event("error", "vanished at frame 2048", frame=2048)
100    last = json.loads(open(path, encoding="utf-8").read().splitlines()[-1])
101    assert last["event"] == "error" and last["frame"] == 2048
102    assert last["gpu"]["temp_c"] == 85.0 # the machine state at the moment of death is
captured
103
104 def test_rotation_caps_size_keeps_header_and_recent(tmp_path, monkeypatch):
105     _no_gpu_name(monkeypatch)
106     path = str(tmp_path / "rt.jsonl")
107     clk = _Clock()
108     t = rt.RunTelemetry(path, max_bytes=600, gpu_fn=_fake_gpu, sys_fn=_fake_sys,
clock=clk)
109     for i in range(200):
110         clk.tick(1)
111         t.snapshot(done=i, total=200, phase="big")
112     data = open(path, encoding="utf-8").read()
113     assert len(data.encode("utf-8")) <= 600 # capped by construction
114     lines = data.splitlines()
115     assert json.loads(lines[0])["meta"] # header preserved
116     assert json.loads(lines[-1])["done"] == 199 # most-recent kept
117     assert len(lines) < 200 # oldest rotated out
118
119
120 def test_collector_failure_is_graceful(tmp_path, monkeypatch):
121     _no_gpu_name(monkeypatch)
122
123     def _boom():
124         raise RuntimeError("nvidia-smi exploded")
125
126     path = str(tmp_path / "rt.jsonl")
127     t = rt.RunTelemetry(path, gpu_fn=_boom, sys_fn=lambda: None, clock=_Clock())
128     rec = t.snapshot(done=1, total=10) # must not raise
129     assert "gpu" not in rec and "sys" not in rec
130

```

14. user (2026-06-14T08:23:33.345Z)

Found 1 file

C:\Users\avidu\Projects\bhi-telwire\tests\test_fusion_compare.py

15. user (2026-06-14T08:23:36.704Z)

```
1: ""GPU-free tests for the fusion-P3 measurement (fusion_golden extractor + fusion_compare
LOV0).
73: def test_extract_video_assembles_shuttle_and_person(monkeypatch):
74:     from backend.eval import fusion_golden
82:     monkeypatch.setattr(fusion_golden.distill_local, "build_candidates",
85:     monkeypatch.setattr(fusion_golden, "load_golden_gts", lambda labels: [(0.0, 10.0)])
95:     data = fusion_golden.extract_video(spec, detector=_StubDetector(), frame_skip=3, iou=0.5)
108: def test_fusion_golden_run_is_resumable(tmp_path, monkeypatch):
110:     from backend.eval import fusion_golden
130:     monkeypatch.setattr(fusion_golden, "extract_video", _stub)
132:     fusion_golden.run(manifest_path, out_dir, fresh=False)
136:     fusion_golden.run(manifest_path, out_dir, fresh=True)
140: def test_fusion_golden_smallest_first(tmp_path, monkeypatch):
142:     from backend.eval import fusion_golden
161:     monkeypatch.setattr(fusion_golden, "extract_video", _stub)
163:     fusion_golden.run(mp, out_dir, smallest_first=True)
167:     fusion_golden.run(mp, out_dir, smallest_first=False, fresh=True)
172:     from backend.eval.fusion_golden import _derive_video_path
```

16. user (2026-06-14T08:23:39.406Z)

```
73     def test_extract_video_assembles_shuttle_and_person(monkeypatch):
74         from backend.eval import fusion_golden
75         from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
76
77         intervals = [(0.0, 10.0), (40.0, 41.0)]
78         x_shuttle = [[10.0, 0.3, 0.2, 0.5, 4.0], [1.0, 0.1, 0.05, 0.2, 0.0]]
79         points = [SimpleNamespace(frame=i, visible=True, x=500.0, y=(100.0 if i % 2 else
900.0))]
80         for i in range(10)]
81
82         monkeypatch.setattr(fusion_golden.distill_local, "build_candidates",
83                             lambda traj, fps, fw: (intervals, x_shuttle))
84         monkeypatch.setattr(TrackNetRunner, "parse_trajectory_csv", staticmethod(lambda
traj: points))
85         monkeypatch.setattr(fusion_golden, "load_golden_gts", lambda labels: [(0.0, 10.0)])
86
87         class _StubDetector:
88             def detections_over_windows(self, video, windows, frame_skip, progress_cb=None):
89                 return [{"frames": {sf: [(1, (10.0, 10.0, 30.0, 40.0))]},
90                                "fps": 30.0, "frame_width": 1920.0, "frame_height": 1080.0}
91                         for (sf, ef) in windows]
92
93         spec = {"name": "t", "trajectory": "bar.csv", "fps": 30.0, "frame_width": 1920.0,
94                "labels": "foo.rallies.csv"}
95         data = fusion_golden.extract_video(spec, detector=_StubDetector(), frame_skip=3,
iou=0.5)
96
97         assert data["name"] == "t"
98         assert data["gts"] == [(0.0, 10.0)]
99         assert len(data["candidates"]) == 2
100        c0 = data["candidates"][0]
101        assert set(c0["shuttle"]) == set(SHUTTLE_FEATURE_NAMES)
102        assert set(c0["person"]) == set(PERSON_FEATURE_NAMES)
103        assert c0["label"] == 1 # (0,10) matches the gt -> positive
104        assert data["candidates"][1]["label"] == 0 # (40,41) -> negative
105        assert all(isinstance(v, float) for v in c0["person"].values())
106
107
108    def test_fusion_golden_run_is_resumable(tmp_path, monkeypatch):
109        """run() skips a video whose JSON already exists (resume a long run); --fresh redoes
all."""
110        from backend.eval import fusion_golden
```

```

111     out_dir = str(tmp_path / "out")
112     os.makedirs(out_dir)
113     manifest = [
114         {"name": "alpha", "trajectory": "a.csv", "fps": 30.0, "frame_width": 1920.0,
"labels": "a.rallies.csv"},
115         {"name": "beta", "trajectory": "b.csv", "fps": 30.0, "frame_width": 1920.0,
"labels": "b.rallies.csv"},
116     ]
117     manifest_path = str(tmp_path / "m.json")
118     with open(manifest_path, "w", encoding="utf-8") as f:
119         json.dump(manifest, f)
120     # a prior run already produced alpha's JSON
121     with open(os.path.join(out_dir, "alpha_golden_features.json"), "w",
encoding="utf-8") as f:
122         json.dump({"name": "alpha", "fps": 30.0, "frame_width": 1920.0, "candidates":
[], "gts": []}, f)
123
124     extracted = []
125
126     def _stub(spec, **kw):
127         extracted.append(spec["name"])
128         return {"name": spec["name"], "fps": 30.0, "frame_width": 1920.0, "candidates":
[], "gts": []}
129
130     monkeypatch.setattr(fusion_golden, "extract_video", _stub)
131
132     fusion_golden.run(manifest_path, out_dir, fresh=False)
133     assert extracted == ["beta"] # alpha skipped (already extracted)
134
135     extracted.clear()
136     fusion_golden.run(manifest_path, out_dir, fresh=True)
137     assert extracted == ["alpha", "beta"] # --fresh redoes both
138
139
140     def test_fusion_golden_smallest_first(tmp_path, monkeypatch):
141         """--smallest-first orders by trajectory size; default keeps manifest order."""
142         from backend.eval import fusion_golden
143         out_dir = str(tmp_path / "out")
144         os.makedirs(out_dir)
145         (tmp_path / "big.csv").write_text("x" * 2000)
146         (tmp_path / "small.csv").write_text("x" * 20)
147         manifest = [
148             {"name": "big", "trajectory": str(tmp_path / "big.csv"), "fps": 30.0,
"frame_width": 1920.0, "labels": "b.rallies.csv"},
149             {"name": "small", "trajectory": str(tmp_path / "small.csv"), "fps": 30.0,
"frame_width": 1920.0, "labels": "s.rallies.csv"},
150         ]
151         mp = str(tmp_path / "m.json")
152         with open(mp, "w", encoding="utf-8") as f:
153             json.dump(manifest, f)
154
155         order = []
156
157         def _stub(spec, **kw):
158             order.append(spec["name"])
159             return {"name": spec["name"], "fps": 30.0, "frame_width": 1920.0, "candidates":
[], "gts": []}
160
161         monkeypatch.setattr(fusion_golden, "extract_video", _stub)
162
163         fusion_golden.run(mp, out_dir, smallest_first=True)
164         assert order == ["small", "big"] # smaller trajectory first
165
166         order.clear()
167         fusion_golden.run(mp, out_dir, smallest_first=False, fresh=True)

```

```
168         assert order == ["big", "small"]           # manifest order preserved
169
170
171     def test_derive_video_path():
172         from backend.eval.fusion_golden import _derive_video_path
```

17. assistant (2026-06-14T08:23:42.233Z)

I now understand the structure. Note the resumability test stubs `extract\_video` with `def \_stub(spec, \*\*kw)` ? so I must pass `telemetry` as a keyword arg into `extract\_video` for the stub to accept it via `\*\*kw`. Now let me implement the wiring.

18. user (2026-06-14T08:23:45.587Z)

The file C:/Users/avidu/Projects/bhi-telwire/backend/eval/fusion_golden.py has been updated successfully. (file state is current in your context ? no need to Read it back)

19. user (2026-06-14T08:23:54.009Z)

The file C:/Users/avidu/Projects/bhi-telwire/backend/eval/fusion_golden.py has been updated successfully. (file state is current in your context ? no need to Read it back)

20. user (2026-06-14T08:24:00.605Z)

The file C:/Users/avidu/Projects/bhi-telwire/backend/eval/fusion_golden.py has been updated successfully. (file state is current in your context ? no need to Read it back)

21. assistant (2026-06-14T08:24:04.018Z)

Now wire telemetry into `run()`. Let me update the loop body with the construction, events, and try/except.

22. user (2026-06-14T08:24:16.733Z)

The file C:/Users/avidu/Projects/bhi-telwire/backend/eval/fusion_golden.py has been updated successfully. (file state is current in your context ? no need to Read it back)

23. assistant (2026-06-14T08:24:20.731Z)

Now let me write the test file. Let me check the conftest/import setup used by tests.

24. user (2026-06-14T08:24:21.866Z)

```
1     """GPU-free tests for the fusion-P3 measurement (fusion_golden extractor +
fusion_compare LOV0).
2
3     The LOV0 driver is exercised on a synthetic 6-video set where the SHUTTLE features are
4     non-separating (constant) but a PERSON feature cleanly separates rallies ? so the
harness must
5     report a positive fusion ?F1. The extractor is exercised with a stubbed detector (no
GPU).
6     """
7     import json
8     import os
9     from types import SimpleNamespace
10
11     import pytest
12
13     from backend.eval import fusion_compare
14     from backend.eval.fusion_compare import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES
15
16
17     def _write_video(d, name):
```

```

18         """A video whose shuttle features are identical for pos/neg (no signal) but whose
19         players_in_court separates rallies (4) from junk (0)."""
20         cands = []

```

25. assistant (2026-06-14T08:24:25.931Z)

Now I'll write the telemetry test file. The key tests: (1) `run()` writes ``run_telemetry.jsonl`` with a meta line + ``video_start`/`video_done`` events; (2) graceful degradation ? a raising gpu collector doesn't break `run()`.

I'll use ``RunTelemetry``'s injectable ``gpu_fn`/`sys_fn`/`clock`` for determinism. Since ``run()`` constructs ``RunTelemetry`` itself (no injection point), I'll monkeypatch ``fusion_golden.RunTelemetry`` with a partial/factory that injects deterministic collectors.

26. assistant (2026-06-14T08:24:53.832Z)

"""GPU-free tests for the run-telemetry wire-in to the fusion_golden extractor.

``run()`` builds a ``RunTelemetry`` black box in ``out_dir`` and emits `video_start / video_done` events around each `extract_video`; a hard kill during a long GPU run thus leaves a forensic runlog.

These

tests stub `extract_video` (no GPU) and inject deterministic collectors + clock via a patched `RunTelemetry` factory, so the JSONL content is asserted with no real `nvidia-smi / psutil / wall-clock`.

A telemetry-collector failure must NOT break extraction (graceful degradation).

"""

```
import functools
```

```
import json
```

```
import os
```

```
from backend.eval import fusion_golden
```

```
from backend.utils import run_telemetry as rt
```

```
class _Clock:
```

```
    def __init__(self, t=1000.0):
        self.t = t
```

```
    def __call__(self):
        return self.t
```

```
    def tick(self, dt):
        self.t += dt
```

```
def _fake_gpu():
```

```
    return {"temp_c": 70.0, "util_pct": 80.0, "throttled": False}
```

```
def _fake_sys():
```

```
    return {"cpu_pct": 50.0, "ram_used_gb": 21.0}
```

```
def _patch_telemetry(monkeypatch, gpu_fn=_fake_gpu, sys_fn=_fake_sys):
```

```
    """Make run()'s `RunTelemetry(...)` hermetic: inject deterministic collectors + clock and a
    fixed GPU-name header (no real nvidia-smi). Returns nothing; run() builds the instance
    itself."""
```

```
    monkeypatch.setattr(rt, "gpu_name", lambda: "FakeGPU")
```

```
    factory = functools.partial(rt.RunTelemetry, gpu_fn=gpu_fn, sys_fn=sys_fn, clock=_Clock())
```

```
    monkeypatch.setattr(fusion_golden, "RunTelemetry", factory)
```

```
def _write_manifest(tmp_path, names):
```

```
    manifest = [
        {"name": n, "trajectory": f"{n}.csv", "fps": 30.0, "frame_width": 1920.0,
```

```

        "labels": f"{n}.rallies.csv"}
    for n in names
]
mp = str(tmp_path / "m.json")
with open(mp, "w", encoding="utf-8") as f:
    json.dump(manifest, f)
return mp

def _read_lines(path):
    with open(path, encoding="utf-8") as f:
        return [ln for ln in f.read().splitlines() if ln]

def test_run_writes_telemetry_with_start_and_done_events(tmp_path, monkeypatch):
    """run() drops run_telemetry.jsonl in out_dir: a meta header + video_start/video_done
    lines."""
    _patch_telemetry(monkeypatch)
    out_dir = str(tmp_path / "out")
    os.makedirs(out_dir)

    def _stub(spec, **kw):
        # A real telemetry instance is threaded through; drive one snapshot from inside
        # extraction.
        telem = kw.get("telemetry")
        if telem is not None:
            telem.snapshot(5, 10, phase=spec["name"])
        return {"name": spec["name"], "fps": 30.0, "frame_width": 1920.0,
                "candidates": [{"label": 1}, {"label": 0}], "gts": []}

    monkeypatch.setattr(fusion_golden, "extract_video", _stub)

    fusion_golden.run(_write_manifest(tmp_path, ["alpha", "beta"]), out_dir)

    rt_path = os.path.join(out_dir, "run_telemetry.jsonl")
    assert os.path.isfile(rt_path)
    lines = _read_lines(rt_path)
    recs = [json.loads(ln) for ln in lines]
    # meta header is always the first line.
    assert recs[0]["meta"]["label"] == "fusion_golden"
    events = [r for r in recs if "event" in r]
    starts = [r for r in events if r["event"] == "video_start"]
    dones = [r for r in events if r["event"] == "video_done"]
    assert {r["msg"] for r in starts} == {"alpha", "beta"}
    # video_done carries the per-video window/positive counts for the post-mortem.
    done_by = {r["msg"]: r for r in dones}
    assert done_by["alpha"]["windows"] == 2 and done_by["alpha"]["positives"] == 1
    # health is attached to every event (the machine state at each milestone).
    assert starts[0]["gpu"]["temp_c"] == 70.0 and starts[0]["sys"]["cpu_pct"] == 50.0
    # every body line is valid JSON.
    for ln in lines:
        json.loads(ln)

def test_run_records_error_event_then_reraises(tmp_path, monkeypatch):
    """A death inside extract_video records an `error` event (with machine state) then re-
    raises."""
    _patch_telemetry(monkeypatch)
    out_dir = str(tmp_path / "out")
    os.makedirs(out_dir)

    def _boom(spec, **kw):
        raise RuntimeError("GPU vanished")

    monkeypatch.setattr(fusion_golden, "extract_video", _boom)

```



```

    raised = False
    try:
        fusion_golden.run(_write_manifest(tmp_path, ["alpha"]), out_dir)
    except RuntimeError as e:
        raised = True
        assert "GPU vanished" in str(e)
    assert raised, "extract_video failure must propagate"

    recs = [json.loads(ln) for ln in _read_lines(os.path.join(out_dir, "run_telemetry.jsonl"))]
    errs = [r for r in recs if r.get("event") == "error"]
    assert len(errs) == 1
    assert errs[0]["msg"] == "alpha" and "GPU vanished" in errs[0]["err"]
    assert errs[0]["gpu"]["temp_c"] == 70.0 # machine state captured at the moment of death

def test_telemetry_collector_failure_does_not_break_run(tmp_path, monkeypatch):
    """A raising GPU collector degrades gracefully ? extraction still completes and writes
    output."""
    def _exploding_gpu():
        raise RuntimeError("nvidia-smi exploded")

    _patch_telemetry(monkeypatch, gpu_fn=_exploding_gpu, sys_fn=lambda: None)
    out_dir = str(tmp_path / "out")
    os.makedirs(out_dir)

    def _stub(spec, **kw):
        telem = kw.get("telemetry")
        if telem is not None:
            telem.snapshot(1, 10, phase=spec["name"]) # must not raise despite the bad
collector
            return {"name": spec["name"], "fps": 30.0, "frame_width": 1920.0,
                    "candidates": [], "gts": []}

    monkeypatch.setattr(fusion_golden, "extract_video", _stub)

    written = fusion_golden.run(_write_manifest(tmp_path, ["alpha"]), out_dir)

    assert written == [os.path.join(out_dir, "alpha_golden_features.json")]
    assert os.path.isfile(written[0])
    recs = [json.loads(ln) for ln in _read_lines(os.path.join(out_dir, "run_telemetry.jsonl"))]
    # The run survived; events still recorded, just without a gpu/sys block.
    assert any(r.get("event") == "video_done" for r in recs)
    assert all("gpu" not in r for r in recs if "event" in r)

def test_run_survives_telemetry_construction_failure(tmp_path, monkeypatch):
    """If RunTelemetry construction itself raises, run() must still extract (telemetry is best-
    effort)."""
    def _boom_ctor(*a, **k):
        raise RuntimeError("cannot build telemetry")

    monkeypatch.setattr(fusion_golden, "RunTelemetry", _boom_ctor)
    out_dir = str(tmp_path / "out")
    os.makedirs(out_dir)

    def _stub(spec, **kw):
        assert kw.get("telemetry") is None # graceful fallback to no telemetry
        return {"name": spec["name"], "fps": 30.0, "frame_width": 1920.0,
                "candidates": [], "gts": []}

    monkeypatch.setattr(fusion_golden, "extract_video", _stub)

    written = fusion_golden.run(_write_manifest(tmp_path, ["alpha"]), out_dir)
    assert written == [os.path.join(out_dir, "alpha_golden_features.json")]

```

27. user (2026-06-14T08:24:53.853Z)

File created successfully at: C:/Users/avidu/Projects/bhi-telwire/tests/test_fusion_golden_telemetry.py (file state is current in your context ? no need to Read it back)

28. assistant (2026-06-14T08:24:57.042Z)

Now let me run the verification commands. First check the Python environment.

29. user (2026-06-14T08:24:58.324Z)

All checks passed!

30. user (2026-06-14T08:25:46.255Z)

backend\sports__init__.py:7: note: By default the bodies of untyped functions are not checked, consider using --check-untyped-defs [annotation-unchecked]
backend\providers__init__.py:7: note: By default the bodies of untyped functions are not checked, consider using --check-untyped-defs [annotation-unchecked]
Success: no issues found in 1 source file

31. user (2026-06-14T08:25:54.830Z)

..... [100%]
11 passed in 3.57s

32. assistant (2026-06-14T08:25:58.507Z)

All green: ruff passes, mypy clean, 11 tests pass. Let me verify the run_telemetry test suite still passes too (sanity) and review the final diff before committing.

33. user (2026-06-14T08:26:01.383Z)

..... [100%]
9 passed in 0.27s

34. user (2026-06-14T08:26:01.614Z)

backend/eval/fusion_golden.py | 28 ++++++-----
1 file changed, 26 insertions(+), 2 deletions(-)

35. assistant (2026-06-14T08:26:04.232Z)

Now let me verify branch safety before committing, then commit with explicit paths.

36. user (2026-06-14T08:26:05.997Z)

feat/telemetry-wire-fusion-golden
ec5bbe0 chore(eval): track experiment_leaderboard.json (fusion P3/P4 honest LOV0 records) (#139)
--- commits ahead of origin/master ---
--- status ---
M backend/eval/fusion_golden.py
?? tests/test_fusion_golden_telemetry.py

37. assistant (2026-06-14T08:26:08.888Z)

Clean: HEAD is at origin/master tip, no sibling commits, only my two paths changed. Let me stage explicitly and commit.

38. user (2026-06-14T08:26:09.916Z)

warning: in the working copy of 'tests/test_fusion_golden_telemetry.py', LF will be replaced by

```
CRLF the next time Git touches it
M backend/eval/fusion_golden.py
A tests/test_fusion_golden_telemetry.py
```

39. user (2026-06-14T08:26:20.037Z)

```
[feat/telemetry-wire-fusion-golden a8710ca] feat(eval): wire run-telemetry black box into
fusion_golden extractor
2 files changed, 196 insertions(+), 2 deletions(-)
create mode 100644 tests/test_fusion_golden_telemetry.py
```

40. user (2026-06-14T08:26:22.859Z)

a8710ca feat(eval): wire run-telemetry black box into fusion_golden extractor

41. assistant (2026-06-14T08:26:24.935Z)

Only my commit. Now push and create the PR.

42. user (2026-06-14T08:26:28.072Z)

```
remote:
remote: Create a pull request for 'feat/telemetry-wire-fusion-golden' on GitHub by visiting:
remote:      https://github.com/avidullu/badminton-highlight-indexer/pull/new/feat/telemetry-
wire-fusion-golden
remote:
branch 'feat/telemetry-wire-fusion-golden' set up to track 'origin/feat/telemetry-wire-fusion-
golden'.
To https://github.com/avidullu/badminton-highlight-indexer.git
* [new branch]      feat/telemetry-wire-fusion-golden -> feat/telemetry-wire-fusion-golden
```

43. user (2026-06-14T08:26:44.497Z)

<https://github.com/avidullu/badminton-highlight-indexer/pull/140>

44. assistant (2026-06-14T08:26:47.351Z)

PR created. Let me confirm the PR number and final state.

45. user (2026-06-14T08:26:49.576Z)

```
{"baseRefName":"master","headRefName":"feat/telemetry-wire-fusion-
golden","number":140,"state":"OPEN","url":"https://github.com/avidullu/badminton-highlight-
indexer/pull/140"}
```

46. assistant (2026-06-14T08:27:00.361Z)

Done. The PR is open and left for review (not merged).

Summary

Wired the run-telemetry black box (`backend/utils/run\_telemetry.py`, #134) into the `fusion\_golden` extractor so long GPU runs auto-keep a forensic ~10KB rotating runlog. Additive, default-ON, fully graceful.

PR: #140 ? <https://github.com/avidullu/badminton-highlight-indexer/pull/140> (base `master`, OPEN, not merged)

Files changed:

```
- `backend/eval/fusion_golden.py` ? imports `RunTelemetry`; `run()` builds
`RunTelemetry(out_dir/run_telemetry.jsonl, label="fusion_golden")` (construction guarded so it
can never raise), emits `video_start` before each extract, `video_done` (windows/positives) on
success, and an `error` event (machine state at death) in a try/except that re-raises.
```

`extract\_video(..., telemetry=None)` snapshots `done/total` from the existing per-frame progress callback.

- `tests/test\_fusion\_golden\_telemetry.py` (new) ? GPU-free; injects deterministic `gpu\_fn`/`sys\_fn`/`clock` via a patched `RunTelemetry` factory. Covers: JSONL meta+start/done events with counts, error-event-then-reraise, raising-collector graceful degradation, and failed-construction fallback.

****Verification (exact results, run in the worktree):****

- `python -m ruff check backend/eval/fusion\_golden.py tests/test\_fusion\_golden\_telemetry.py` ?
`All checks passed!`
- `python -m mypy backend/eval/fusion\_golden.py` ? `Success: no issues found in 1 source file`
- `python -m pytest tests/test\_fusion\_compare.py tests/test\_fusion\_golden\_telemetry.py -q` ? `11 passed in 3.57s`
- (sanity) `python -m pytest tests/test\_run\_telemetry.py -q` ? `9 passed` ? no regression.
Existing `test\_fusion\_golden\_run\_is\_resumable` stays green (its `extract\_video` stub uses `\*\*kw`, so the new `telemetry=` kwarg flows through harmlessly).

****Branch safety:**** worked in an isolated worktree, branched from `origin/master`; `git log --oneline origin/master..HEAD` listed only my single commit; staged explicit paths only.

****Worktree path:**** `C:/Users/avidu/Projects/bhi-telwire` (left in place as requested).

Notes: I did not touch `scratch/extract\_until\_done.ps1` ? it's gitignored and the wiring is internal to `run()`, so no change was needed there. No audio work, no other changes.